CHAPTER 4

Subtyping and Variance

You pass a string literal, of type `&'static str`, to a function that expects a `&'a str` for some short `'a`, and you never give it a thought. Yet something non-trivial happens there. Rust has no class hierarchy, no `i32` that is also an `i64`; in general a value of one type cannot be used where another type is expected. Here it can: a reference that is valid for the whole program is accepted where a reference valid for a shorter region is needed. That is *subtyping*, and in Rust it comes from lifetimes.

As soon as lifetimes have subtypes, every type that contains a lifetime raises a question. If a `&'static str` can be used as a `&'a str`, can a `Vec<&'static str>` be used as a `Vec<&'a str>`? Can a `fn(&'static str)` be used as a `fn(&'a str)`? Can a `&mut &'static str` be used as a `&mut &'a str`? The answers are yes, no, and absolutely not, and the rules that give them are called *variance*. They are not an academic curiosity. The "absolutely not" for `&mut` is what keeps a dangling reference out of safe code; variance notes appear in borrow-checker diagnostics ("mutable references are invariant over their type parameter"); and variance is the reason a method taking `&'a mut self` on a `Thing<'a>` can lock a value for the rest of its life.

In this chapter you will learn:

- how the outlives relation orders regions, and how that order makes `&'long T` a subtype of `&'short T`;
- what covariant, contravariant and invariant mean, and the variance of the standard types, from `&'a T` and `Box<T>` to `Cell<T>`, `fn(T) -> U`, `dyn Trait + 'a` and `PhantomData<T>`;
- how to make the compiler confirm a variance with a *witness* function;
- why `&mut T` and `Cell<T>` must be invariant in `T`, through the dangling reference that covariance would allow;
- why function arguments are contravariant, with a callback that accepts only `'static` strings;
- how the compiler computes the variance of your own types, why `&'a mut T<'a>` locks a value, and how to read variance in diagnostics.

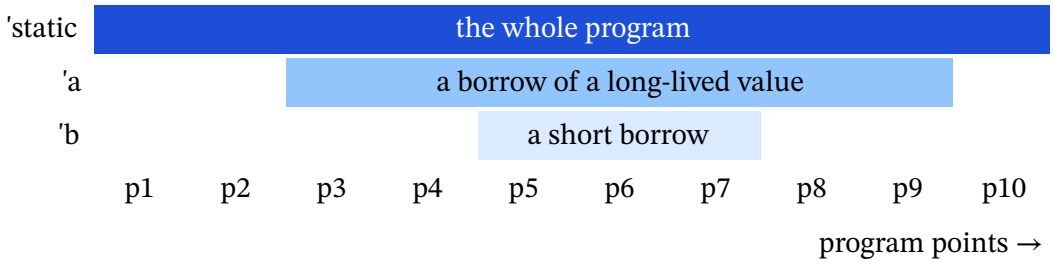
4.1 The outlives order

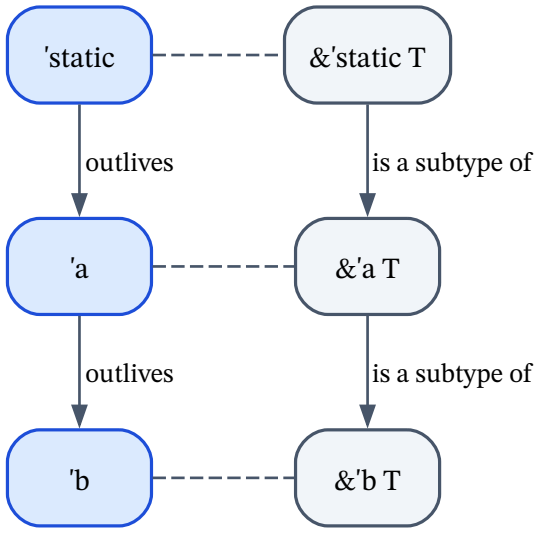
Chapter 1 described a lifetime as a *region*: a set of points in the control flow of a function body (a simplification of what rustc computes, but a faithful one for everything in this chapter). Regions can be compared. We say that `'a` *outlives* `'b`, written `'a: 'b`, when `'a` contains every point of `'b`:

a reference that is valid throughout `'a` is in particular valid throughout `'b`. The relation has the properties you would expect of "is at least as long as":

- every region outlives itself (`'a: 'a`);
- it is transitive: if `'a: 'b` and `'b: 'c`, then `'a: 'c`;
- `'static`, the region of the whole program, outlives every other region, so it is the largest one.

It is not a total order. Two borrows in different branches of an `if`, or in two sibling blocks, have regions that do not contain each other, and then neither outlives the other. Inside a function body the borrow checker computes the regions and checks the outlives relations that the code needs. In a signature, you can demand one with a bound such as `'long: 'short`, which every caller must then satisfy.





The outlives order on regions: `'static` contains `'a`, which contains `'b`, and the order carries over to subtyping: `&'static T` is a subtype of `&'a T`, which is a subtype of `&'b T`.

4.2 Subtyping: a longer borrow is a shorter one

A type `S` is a *subtype* of a type `T` when a value of type `S` can be used wherever a value of type `T` is expected. The usual notation is `S <: T`; it is not Rust syntax, but it is convenient in prose. Rust has exactly one source of subtyping, the outlives order:

If `'long: 'short`, then `&'long T <: &'short T`.

A reference that is valid over a large region is valid over any part of it. Read the direction carefully, because it is easy to get backwards: the *longer* lifetime gives the *subtype*. `&'static T` is a subtype of every `&'a T`, because it carries the strongest guarantee. A subtype is the type with more guarantees, which is why it can stand in for the type with fewer.

Here is the everyday case, from `code/examples/ch04_covariance.rs`:

```
fn pick<'a>(preferred: &'a str, fallback: &'a str) -> &'a str {
    if preferred.is_empty() {
        fallback
    } else {
        preferred
    }
}
```

```
let fallback: &'static str = "anonymous";
let typed = String::from("ferris");
// `'a` becomes (at most) the region of the borrow of `typed`; the
// `&'static str` is used as a `&'a str`.
println!("{}", pick(&typed, fallback));
```

```
ferris
```

`pick` asks for two arguments of one type, `&'a str`. The call passes a borrow of `typed`, which is valid only while `typed` lives, and `fallback`, which is valid for the whole program. The compiler picks for `'a` a region no larger than the borrow of `typed`, and uses subtyping to accept `fallback` as a `&'a str`. This is sometimes described as the compiler "shrinking" the lifetime of `fallback`. Nothing happens at run time: it is the same pointer, and lifetimes do not exist in the compiled program. Subtyping is purely a fact the type checker uses to accept the program.

Where can a subtype be used in place of its supertype? Anywhere a value flows into a place of a known type: a function argument, a `let` with a type annotation, an assignment, a return value, a struct field. Rust's other kind of subtyping, between higher-ranked function types such as `for<'a> fn(&'a str)` and `fn(&'x str)`, follows the same idea and belongs to Chapter 9.

4.3 Variance: how subtyping passes through types

Subtyping between two references raises the question from the introduction: what about types that *contain* references? The answer depends on the type, and it is called its *variance*. Take a generic type `F<X>` and two types with `S <: T`. Then `F` is, in its parameter `X`:

- **covariant** if `F<S> <: F<T>`: the subtyping passes through in the same direction;
- **contravariant** if `F<T> <: F<S>`: the subtyping passes through reversed;
- **invariant** if neither holds (unless `S` and `T` are the same type): no subtyping passes through.

A type with several parameters has a variance in each, and lifetime parameters count: `&'a T` has a variance in `'a` and another in `T`. These are the variances of the standard types:

Type	Variance in `'a`	Variance in `T` (and `U`)
`&'a T`	covariant	covariant
`&'a mut T`	covariant	**invariant**
`Box<T>`, `Vec<T>`, `Option<T>`, `Rc<T>`, `Arc<T>`		covariant
`Cell<T>`, `RefCell<T>`, `UnsafeCell<T>`, `Mutex<T>`		**invariant**
`*const T` / `*mut T`		covariant / **invariant**
`fn(T) -> U`		**contravariant** in `T`, covariant in `U`
`dyn Trait<T> + 'a`	covariant	**invariant**
`PhantomData<T>`		covariant

The Reference (https://doc.rust-lang.org/reference/subtyping.html) has the same table, with the two halves of the function row as separate entries, `fn() -> T` covariant and `fn(T) -> ()` contravariant. The Rustonomicon's chapter on subtyping and variance (https://doc.rust-lang.org/nomicon/subtyping.html) is the page the compiler itself points to in its variance notes, as we will see.

Making the compiler prove it

You do not have to take the table on trust. A function whose body is just its argument, and whose signature converts one type into another, compiles only if the argument type is a subtype of the return type. Such a function is a *witness*: if it compiles, the subtyping holds. Here are witnesses for the first rows, from `code/examples/ch04_variance_witnesses.rs`:

```
// In every witness, 'long: 'short ("'long outlives 'short").

// `&'a T` is covariant in 'a...
fn shrink_ref<'long: 'short, 'short>(r: &'long str) -> &'short str {
    r
}

// ...and in T (here T = &'long str, behind a reference of some lifetime 'r).
fn shrink_inner<'long: 'short, 'short, 'r>(r: &'r &'long str) -> &'r &'short str {
    r
}

// Owning containers are covariant in their element type.
fn shrink_vec<'long: 'short, 'short>(v: Vec<&'long str>) -> Vec<&'short str> {
    v
}

// `&'a mut T` is covariant in 'a: the loan can be shortened...
fn shrink_loan<'long: 'short, 'short, 'v>(
    r: &'long mut &'v str,
) -> &'short mut &'v str {
    r
}
// ...but its T (here `&'v str`) must stay exactly as it is.
```

Read each signature as a claim. `shrink_vec` says: "given that `'long` outlives `'short`, a `Vec<&'long str>` is a `Vec<&'short str>`." The body adds nothing, so the compiler accepts the function only if `Vec` is covariant in its element type. `shrink_inner` makes the same claim for the `T` of a shared reference, and `shrink_loan` for the *lifetime* of a mutable reference, not its `T`. The same file has a witness for each

of the other covariant rows (`Box`, `Option`, `Rc`, `Arc`, `*const T`, the result of `fn() -> T`, the bound of `dyn Display + 'a`, and `PhantomData`), and runs them all:

```
shrink_ref:       literal
shrink_inner:     inner
shrink_vec:       ["a", "b"]
shrink_loan:      slot
shrink_box:       boxed
shrink_option:    Some("some")
shrink_rc:        shared
shrink_arc:       atomic
shrink_const_ptr: false
shrink_fn_result: hello
shrink_dyn_bound: 42
grow_fn_arg:      4
shrink_twice:     a literal
```

For the rows marked invariant, the opposite happens: the witness does not compile. We will meet those errors in a moment, because they are the important part of the table.

4.4 Covariance in action

Covariance is what makes ordinary code work without a second thought. The rest of `ch04_covariance.rs` moves values with `'static` contents into variables whose types have shorter lifetimes:

```
let defaults: Vec<&'static str> = vec!["red", "green"];
let custom = String::from("teal");
let mut palette: Vec<&str> = defaults; // Vec<&'static str> used as Vec<&'p str>
palette.push(&custom);
println!("{palette:?}");

let boxed: Box<&'static str> = Box::new("static");
let mut current: Box<&str> = boxed; // Box<&'static str> used as Box<&'c str>
println!("{current}");
current = Box::new(&custom);
println!("{current}");
```

```
["red", "green", "teal"]
static
teal
```

`defaults` is a `Vec<&'static str>`. Moving it into `palette`, a `Vec<&'p str>` with a shorter `'p`, is allowed because `Vec` is covariant; then `palette` can receive a borrow of the local `custom`. The same happens with `Box<&'static str>` and `current`.

Why is this safe, when a `Vec` can be written to? Because after the move nobody holds the vector at the old type any more. The only one who could still expect `'static` strings in it would be someone who can look at it as a `Vec<&'static str>`, and the move took that view away. Owning containers are covariant for this reason: the owner is the only one who can read or write, so it can choose the type it sees the contents at. Writing into a container that someone *else* owns takes a `&mut`, and that is where invariance comes in.

4.5 Invariance: the safety net for writes

Suppose, for a moment, that `&mut T` were covariant in `T`. Here is what would then compile, from `code/compile_fail/ch04_mut_invariance.rs`:

```
fn overwrite<'a>(slot: &mut &'a str, value: &'a str) {
    *slot = value;
}

pub fn caller() {
    let mut name: &'static str = "static";
    {
        let local = String::from("local");
        overwrite(&mut name, &local); // would leave `name` dangling
    }
    println!("{name}");
}
```

`name` has the type `&'static str`, so `&mut name` is a `&mut &'static str`. With covariance, the compiler could view it as a `&mut &'short str`, where `'short` is the region of the borrow of `local`, and the call `overwrite(&mut name, &local)` would type-check with `'a = 'short`. `overwrite` stores the borrow of `local` in `name`. At the closing brace `local` is dropped, and `println!("{name}")` reads freed memory, through a variable whose type still promises a string that lives forever. Every step is locally reasonable; only the covariance assumption is wrong. Rust's actual rule rejects the program:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_mut_invariance.rs:14:30
   |
11 |     let mut name: &'static str = "static";
   |                   ------------ type annotation requires that `local` is borrowed for
`'static`
12 |     {
13 |         let local = String::from("local");
   |             ----- binding `local` declared here
14 |         overwrite(&mut name, &local); // would leave `name` dangling
   |                              ^^^^^^ borrowed value does not live long enough
15 |     }
   |     - `local` dropped here while still borrowed
```

Because `&mut T` is invariant in `T`, the type `&mut &'static str` of the first argument fixes `T = &'static str` exactly, with no room to shorten it, so `'a` must be `'static`. Then the second argument, `&local`, must be a `&'static str`, which it cannot be. The diagnostic is the familiar E0597, and its first label names the root of the requirement: the "type annotation requires that `local` is borrowed for `'static`". The annotation `&'static str` on `name` fixed one end, and invariance carried it, through `overwrite`'s signature, to `local`.

The invariant rows, confirmed

The program `code/compile_fail/ch04_invariant_witnesses.rs` has witnesses for four of the invariant entries of the table, and every one of them fails:

```
pub fn shrink_target<'long: 'short, 'short, 'v>(
    r: &'v mut &'long str,
) -> &'v mut &'short str {
    r
}
```

```
pub fn shrink_cell<'long: 'short, 'short>(c: Cell<&'long str>) -> Cell<&'short str> {
    c
}

pub fn shrink_mut_ptr<'long: 'short, 'short>(p: *mut &'long str) -> *mut &'short str {
    p
}

pub fn shrink_dyn_fn<'long: 'short, 'short>(
    f: Box<dyn Fn(&'long str)>,
) -> Box<dyn Fn(&'short str)> {
    f
}
```

Here is the first error in full:

```
error: lifetime may not live long enough
  --> compile_fail/ch04_invariant_witnesses.rs:11:5
   |
 8 | pub fn shrink_target<'long: 'short, 'short, 'v>(
   |                      -----          ------ lifetime `'short` defined here
   |                      |
   |                      lifetime `'long` defined here
...
11 |     r
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of a mutable reference to `&str`
   = note: mutable references are invariant over their type parameter
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The errors for `Cell` and for the raw pointer have the same shape, and differ only in their notes. For `Cell`:

```
= note: requirement occurs because of the type `Cell<&str>`, which makes the generic
argument `&str` invariant
= note: the struct `Cell<T>` is invariant over the parameter `T`
```

and for `*mut`:

```
= note: requirement occurs because of a mutable pointer to `&str`
= note: mutable pointers are invariant over their type parameter
```

(The fourth error, for `dyn Fn`, comes up in the section on contravariance.) In all of them, the primary message is "lifetime may not live long enough", and the label under the returned value says "function was supposed to return data with lifetime `'long` but it is returning data with lifetime `'short`", which is a roundabout way of saying that the conversion needs the two lifetimes to be *equal*. The bound `'long: 'short` gives one direction; the help asks for the other, `'short: 'long`, which together would make them the same region. (Adding that bound is almost never the fix you want: it removes the difference between the two lifetimes, and with it the point of the function.) Then the notes name the reason: "a mutable reference to `&str`" with "mutable references are invariant over their type parameter"; "the type `Cell<&str>`, which makes the generic argument `&str` invariant" with "the struct `Cell<T>` is invariant over the parameter `T`"; and the same for "mutable pointers". The last help links to

the Rustonomicon's chapter on variance. The file `code/compile_fail/ch04_invariant_cells.rs` checks `RefCell<T>`, `UnsafeCell<T>` and `Mutex<T>` the same way, with the same notes.

`Cell` writes through a shared reference

`Cell<T>` is the clearest case of why invariance is about *writing* rather than about `mut`. A `Cell` can be written through a shared reference (Chapter 12), so the overwrite attack needs no `&mut` at all. The program `code/compile_fail/ch04_cell_invariance.rs` tries:

```
pub fn caller() {
    let label: Cell<&'static str> = Cell::new("static");
    let local = String::from("local");
    let view: &Cell<&str> = &label; // Cell<&'static str> is NOT a Cell<&'short str>
    view.set(&local);
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_cell_invariance.rs:12:14
   |
 9 |     let label: Cell<&'static str> = Cell::new("static");
   |                ------------------ type annotation requires that `local` is borrowed for
`'static`
10 |     let local = String::from("local");
   |         ----- binding `local` declared here
11 |     let view: &Cell<&str> = &label; // Cell<&'static str> is NOT a Cell<&'short str>
12 |     view.set(&local);
   |              ^^^^^^ borrowed value does not live long enough
13 | }
   | - `local` dropped here while still borrowed
```

If `Cell` were covariant, `view` would be a `&Cell<&'short str>` looking at the same cell as `label`, `view.set(&local)` would store a short-lived borrow in it, and `label.get()` would later return it as a `&'static str`. Invariance makes the annotation on `label` fix the type of the cell's contents for everyone, so `set` demands a `&'static str`. Interior mutability is a write through a shared path, and every type built on `UnsafeCell<T>` is invariant in `T` for this reason.

Reading, writing, and both

The principle behind these rules fits in one line: **reading is covariant, writing is contravariant, and both is invariant.** A place you can only read from is a *source* of `T` values; if it produces `&'static str`s, you can certainly treat what it produces as `&'short str`s. A place you can write to is a *sink*; if it accepts `&'short str`s, it certainly accepts `&'static str`s, so the subtyping goes the other way. A `&mut T`, a `Cell<T>` or a `*mut T` is both a source and a sink of `T`, and the only way for both directions to hold is for the types to be equal.

The same principle explains the one covariant entry of `&'a mut T`: its lifetime `'a`. Shortening a loan only gives the exclusive access back sooner, which is always safe; that is `shrink_loan` above. What may not change is the type of what the loan points to.

4.6 Contravariance: function arguments

A function is a sink for its arguments and a source of its result, so the reading and writing principle predicts its variance: covariant in the result, contravariant in the arguments. The contravariant

direction is easiest to see with callbacks. Suppose some code will call a callback with `&'static str` arguments only. A callback that accepts *any* short-lived `&'a str` is perfectly acceptable there: whatever it receives, it can handle. So `fn(&'a str) <: fn(&'static str)`, even though `&'static str <: &'a str`: the function that accepts more is the subtype. The witness from `ch04_variance_witnesses.rs`:

```
// The direction is reversed: a callback for SHORT-lived strings is usable as a
// callback for LONG-lived ones.
fn grow_fn_arg<'long: 'short, 'short>(
    f: fn(&'short str) -> usize,
) -> fn(&'long str) -> usize {
    f
}
```

The opposite substitution, a callback that can only handle `'static` strings offered to code that will call it with a short-lived one, would be a disaster. The compiler rejects it, as `code/compile_fail/ch04_static_callback.rs` shows:

```
fn apply<'a>(callback: fn(&'a str) -> usize, text: &'a str) -> usize {
    callback(text)
}

fn only_static(text: &'static str) -> usize {
    text.len()
}

pub fn caller() -> usize {
    let local = String::from("local");
    apply(only_static, &local)
}
```

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch04_static_callback.rs:16:24
   |
15 |     let local = String::from("local");
   |         ----- binding `local` declared here
16 |     apply(only_static, &local)
   |           ------------  ^^^^^^ borrowed value does not live long enough
   |           |
   |           coercion requires that `local` is borrowed for `'static`
17 | }
   | - `local` dropped here while still borrowed
```

`apply` takes a callback of type `fn(&'a str) -> usize` and a text of type `&'a str`, with one `'a`. The function `only_static` has type `fn(&'static str) -> usize`. For it to be a `fn(&'a str) -> usize`, contravariance requires `&'a str <: &'static str`, which holds only if `'a` is `'static`. So `'a` becomes `'static`, and `&local` would have to be a `&'static str`. The label "coercion requires that `local` is borrowed for `'static`" points at `only_static`: the function item is converted (coerced) to a function pointer there, and that is where the subtyping requirement arose. Imagine that `only_static` stored its argument in a global list of names; with a short-lived string, the list would end up holding a dangling reference.

You will meet contravariance less often than the other two variances, for a reason that Chapter 3 already showed: a function pointer type you write, such as `fn(&str) -> usize`, is higher-ranked, so it accepts every lifetime and needs no subtyping at all. Contravariance matters when the argument's

lifetime is fixed by a surrounding lifetime parameter, as in `apply`, or in a struct like `Handler<'a>` in the next section.

Trait objects behave differently. A `Box<dyn Fn(&'a str)>` looks like a function taking `&'a str`, but `'a` there is a parameter of the trait `Fn`, and the parameters of a trait object are invariant. That is the fourth witness of `ch04_invariant_witnesses.rs`:

```
error: lifetime may not live long enough
  --> compile_fail/ch04_invariant_witnesses.rs:25:5
   |
22 | pub fn shrink_dyn_fn<'long: 'short, 'short>(
   |                      -----          ------ lifetime `'short` defined here
   |                      |
   |                      lifetime `'long` defined here
...
25 |     f
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
```

Note what is missing: no note explains the invariance this time. When a "lifetime may not live long enough" error involves a `dyn Trait<...>`, remember that only the `+ 'a` bound of a trait object is covariant; everything inside the trait's own parameters is invariant.

4.7 The variance of your own types

You never declare the variance of a struct or an enum; the compiler computes it from the fields. For each parameter, it looks at every position in which the parameter occurs, and combines their variances:

- if every occurrence is covariant, the type is covariant in the parameter;
- if every occurrence is contravariant, it is contravariant;
- if the occurrences disagree, or any of them is invariant, it is invariant.

The Reference puts it like this: "The variance of other `struct`, `enum`, and `union` types is decided by looking at the variance of the types of their fields. If the parameter is used in positions with different variances then the parameter is invariant." Nesting composes: a parameter inside a covariant type keeps the variance it has there, a parameter anywhere inside an invariant type is invariant, and a parameter inside a contravariant position is flipped. Three structs from `code/examples/ch04_struct_variance.rs` show the three outcomes:

```rust
// Only read through: covariant in 'src.
struct Tokens<'src> {
    text: &'src str,
    position: usize,
}

// 'a appears only as an argument of a fn pointer: contravariant in 'a.
struct Handler<'a> {
    on_line: fn(&'a str) -> usize,
}

// Written through a shared reference: invariant in 'a.
struct Slot<'a> {
    current: Cell<&'a str>,
}
```

`Tokens` uses `'src` once, in `&'src str`, which is covariant in its lifetime; `position` does not mention `'src`. So `Tokens` is covariant in `'src`. `Handler` uses `'a` only in the argument of a function pointer, a contravariant position, so it is contravariant. `Slot` uses `'a` inside a `Cell`, which is invariant, so it is invariant. The witnesses confirm the first two:

```
// Covariant: Tokens<'long> can be used as Tokens<'short>.
fn shrink_tokens<'long: 'short, 'short>(t: Tokens<'long>) -> Tokens<'short> {
    t
}

// Contravariant: a handler for any short-lived line can handle 'static lines.
fn handler_for_static<'a>(h: Handler<'a>) -> Handler<'static> {
    h
}

// Invariant: the only conversion is to the very same type.
fn same_slot<'a>(s: Slot<'a>) -> Slot<'a> {
    s
}
```

```
rest: "x = 1;"
length: 14
slot: new
```

`handler_for_static` is contravariance in a struct: a handler that accepts lines of any lifetime `'a` can be stored where a handler for `'static` lines is expected. `same_slot` is there for contrast: the identity is the only conversion an invariant type allows. The proof that `Slot` is invariant is a witness that fails, in `code/compile_fail/ch04_struct_invariant.rs`:

```
pub struct Slot<'a> {
    current: Cell<&'a str>,
}

pub fn shrink_slot<'long: 'short, 'short>(s: Slot<'long>) -> Slot<'short> {
    s
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch04_struct_invariant.rs:13:5
   |
12 | pub fn shrink_slot<'long: 'short, 'short>(s: Slot<'long>) -> Slot<'short> {
   |                    -----          ------ lifetime `'short` defined here
   |                    |
   |                    lifetime `'long` defined here
13 |     s
   |     ^ function was supposed to return data with lifetime `'long` but it is returning
data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of the type `Slot<'_>`, which makes the generic
argument `'_` invariant
   = note: the struct `Slot<'a>` is invariant over the parameter `'a`
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The notes report the variance the compiler computed: "the type `Slot<'_>`, which makes the generic argument `'_` invariant", and "the struct `Slot<'a>` is invariant over the parameter `'a`". When you see this note about a type of your own, look for the field that makes it invariant: a `&mut`, a `Cell` or

`RefCell`, a `*mut`, a trait object with the parameter inside, or a parameter used both as an input and as an output of a function pointer.

Two consequences are worth remembering.

- **Variance is part of a type's public interface.** Changing a private field from `&'a str` to `Cell<&'a str>` changes no signature, but it makes the struct invariant, and code elsewhere that relied on shortening its lifetime stops compiling.
- **A parameter that no field uses has no variance at all**, and Rust rejects the struct (E0392) instead of guessing. This happens with types built on raw pointers, where the lifetime the type stands for is not visible in any field. The fix is a field of type `PhantomData<X>`, a zero-sized marker that makes the compiler compute the variance as if the struct contained an `X`: with `PhantomData<&'a T>` the struct behaves like a `&'a T`, covariant in both. Chapter 13 shows how to choose the marker for each variance.

4.8 Invariance in practice: `&'a mut T<'a>`

Invariance has one consequence that every Rust programmer meets sooner or later. Chapter 2 showed the symptom: a method with a receiver of type `&'a mut self` on a type with lifetime parameter `'a` borrows the value for the rest of its life. Here is the mechanism. The program `code/compile_fail/ch04_mut_self_lock.rs` has a collector of words borrowed from some text, with the problematic signature:

```
pub struct Collector<'src> {
    words: Vec<&'src str>,
}

impl<'src> Collector<'src> {
    pub fn add(&'src mut self, line: &'src str) {
        self.words.extend(line.split_whitespace());
    }
}
```

```
pub fn caller() {
    let text = String::from("alpha beta");
    let more = String::from("gamma");
    let mut collector = Collector { words: Vec::new() };
    collector.add(&text);
    collector.add(&more);
}
```

```
error[E0499]: cannot borrow `collector` as mutable more than once at a time
  --> compile_fail/ch04_mut_self_lock.rs:23:5
   |
22 |     collector.add(&text);
   |     --------- first mutable borrow occurs here
23 |     collector.add(&more);
   |     ^^^^^^^^^
   |     |
   |     second mutable borrow occurs here
   |     first borrow later used here
```

Follow the lifetimes. Say `collector` has the type `Collector<'c>`. Because the value's type mentions `'c`, the region `'c` must include every point where `collector` is still going to be used; that is how

the borrow checker keeps the words in the collector valid. The call `collector.add(&text)` creates a mutable borrow `&'m mut Collector<'c>` and passes it where a `&'src mut Collector<'src>` is expected:

1. `&'m mut X` is covariant in `'m`, so the borrow is acceptable if `'m: 'src`: the loan lasts at least as long as `'src`.
2. `&'m mut X` is invariant in `X`, so `Collector<'c>` must *equal* `Collector<'src>`: `'src` is `'c`, with no room to shorten it.
3. Together: `'m: 'c`. The mutable borrow lasts as long as the collector is used.

So the first mutable borrow is still alive at the second call, and the compiler reports E0499, "cannot borrow `collector` as mutable more than once at a time", with the first borrow "later used here" at the second call. Any other use of `collector` after the first `add`, even reading `collector.words.len()`, would be rejected too (as E0502, a shared borrow while a mutable one is alive). Chapter 3 met the same shape when the hypothetical rule "the same lifetime everywhere" turned `push` into `fn push<'a>(list: &'a mut Vec<&'a str>, item: &'a str)`.

The shared version of the same signature is harmless, and the difference is variance again. Here it is in `code/examples/ch04_shared_self_borrow.rs`:

```
struct Collector<'src> {
    words: Vec<&'src str>,
}

impl<'src> Collector<'src> {
    // The right signature: a short `&mut` borrow, and lines that live for 'src.
    fn add(&mut self, line: &'src str) {
        self.words.extend(line.split_whitespace());
    }

    // Over-constrained (`&'src self`), but harmless thanks to covariance.
    fn first(&'src self) -> Option<&'src str> {
        self.words.first().copied()
    }
}
```

```
fn main() {
    let text = String::from("alpha beta");
    let more = String::from("gamma");
    let mut collector = Collector { words: Vec::new() };
    collector.add(&text);
    println!("{:?}", collector.first()); // a shared borrow...
    collector.add(&more); // ...that has already ended: `add` may borrow mutably
    println!("{:?} of {}", collector.first(), collector.words.len());
}
```

```
Some("alpha")
Some("alpha") of 3
```

`first` takes `&'src self`, the same suspicious shape with `&` instead of `&mut`. A `&'m Collector<'c>` is passed where a `&'src Collector<'src>` is expected, and this time `Collector<'c>` only has to be a *subtype* of `Collector<'src>`, which covariance grants for any `'src` shorter than `'c`. The compiler picks a short `'src` for this one call, the shared borrow ends after the `println!`, and `add` can borrow the collector mutably again. The fix for the mutable case is the signature `add` has here: `&mut self` with

its own elided, short lifetime, and `'src` only on the data that really lives that long. Chapter 14 returns to this pattern and the ways to design around it.

4.9 Variance in diagnostics

Variance rarely appears in the primary message of an error. It shows up in the notes, and in a few labels that are easier to read once you know what to look for. All of these come from the programs of this chapter and of Chapter 3:

The diagnostic says	What it means
"requirement occurs because of a mutable reference to `X`" and "mutable references are invariant over their type parameter"	a lifetime inside `X` cannot change, because `X` is behind a `&mut`
"requirement occurs because of a mutable pointer to `X`" and "mutable pointers are invariant over their type parameter"	the same, behind a `*mut`
"requirement occurs because of the type `S<A>`, which makes the generic argument `A` invariant" and "the struct `S<T>` is invariant over the parameter `T`"	a field of `S` (yours, or `Cell`, `RefCell`, `Mutex` in the standard library) makes `S` invariant
"function was supposed to return data with lifetime `'x` but it is returning data with lifetime `'y`" with "consider adding the following bound: `'y: 'x`"	an invariant position needs the two lifetimes to be equal
"type annotation requires that `v` is borrowed for `'static`"	an annotation fixed a lifetime, and invariance carried the requirement to `v`
"coercion requires that `v` is borrowed for `'static`"	a function item became a function pointer whose argument lifetime is fixed

Two habits help. First, when a note mentions invariance, find the `&mut`, the cell or the struct it names, and ask which lifetime inside it is being forced to match another; that is usually the lifetime you should separate. Second, when an error involves a trait object, remember that its trait parameters are invariant even though no note says so.

4.10 For the type-theoretically inclined

The regions of a program, ordered by outlives, form a partial order with `'static` as its greatest element. A type constructor maps regions (or types) to types, and variance says how it treats the order: a covariant constructor preserves it (a monotone map), a contravariant one reverses it, and an invariant one relates only equal arguments. Variances compose like signs: a contravariant position inside another contravariant position is covariant, which is why a function that takes a callback is covariant in the callback's argument:

```
// An argument of an argument: two reversals, so covariant again.
fn shrink_twice<'long: 'short, 'short>(f: fn(fn(&'long str))) -> fn(fn(&'short str)) {
```

```
    f
}
```

`&mut T` puts `T` in a read position (covariant) and a write position (contravariant) at once, and the only variance compatible with both is invariance. The same calculation, done field by field, is how the compiler infers the variance of your structs.

4.11 Common confusions

"**Rust has no subtyping.**" It has no subtyping between structs or traits, but it has subtyping between lifetimes (and between higher-ranked function types, Chapter 9), and through variance that subtyping reaches every type that contains a lifetime. You use it every time you pass a `&'static str` where a shorter `&str` is expected.

"**A longer lifetime is a supertype.**" The other way round: `&'static T` is a subtype of every `&'a T`. The subtype is the type with the stronger guarantee.

"**`&mut T` is invariant.**" In `T`, yes. In its own lifetime it is covariant: shortening a loan is always safe; changing what it points to is not.

"**Invariant means `'static`.**" Invariance only means that a lifetime cannot be adjusted. The `'static` in many errors comes from somewhere else, an annotation or a `'static` value, and invariance carries it to a place where it does not fit.

"**`Box<dyn Fn(&'a str)>` is contravariant, like `fn(&'a str)`.**" The lifetime is a parameter of the trait, and trait parameters of a trait object are invariant. Only the object's `+ 'a` bound is covariant.

"**A coercion is subtyping.**" Converting `&String` to `&str` is a deref coercion: it converts one value, at one place (an argument, a `let` with a type), and it does not pass through other types. Subtyping does, according to variance. The program `code/compile_fail/ch04_no_coercion_in_vec.rs` shows the difference:

```
fn total_len(names: Vec<&str>) -> usize {
    names.iter().map(|name| name.len()).sum()
}

pub fn caller() -> usize {
    let owned = String::from("ferris");
    let one: &str = &owned; // fine: a deref coercion from &String to &str
    let names: Vec<&String> = vec![&owned];
    total_len(names) + one.len()
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch04_no_coercion_in_vec.rs:14:15
   |
14 |     total_len(names) + one.len()
   |     --------- ^^^^^ expected `Vec<&str>`, found `Vec<&String>`
   |     |
   |     arguments to this function are incorrect
   |
   = note: expected struct `Vec<&str>`
              found struct `Vec<&String>`
note: function defined here
```

```
  --> compile_fail/ch04_no_coercion_in_vec.rs:6:4
   |
6  | fn total_len(names: Vec<&str>) -> usize {
   |    ^^^^^^^^^ -----------------
```

A `Vec<&'static str>` can be used as a `Vec<&'a str>` because `Vec` is covariant, but a `Vec<&String>` cannot become a `Vec<&str>`: that would require converting every element, and building a new vector.

"**I can declare the variance I want.**" In Rust the variance of a type is always computed from its fields. To influence it, you change the fields, or add a `PhantomData` marker (Chapter 13).

4.12 Summary

- Regions are ordered by outlives: `'a: 'b` means `'a` contains `'b`. The order is reflexive and transitive, not total, and `'static` is the largest region.
- If `'long: 'short`, then `&'long T` is a subtype of `&'short T`. This is Rust's only subtyping (besides higher-ranked function types), it costs nothing at run time, and it is not a coercion.
- Variance says how subtyping passes through a type: covariant (same direction), contravariant (reversed), invariant (not at all). `&'a T`, `Box`, `Vec`, `Option`, `Rc`, `Arc`, `*const T` and `PhantomData<T>` are covariant; `&'a mut T` is covariant in `'a` and invariant in `T`; `Cell`, `RefCell`, `UnsafeCell`, `Mutex` and `*mut T` are invariant; `fn(T) -> U` is contravariant in `T` and covariant in `U`; `dyn Trait<T> + 'a` is covariant in `'a` and invariant in `T`.
- A witness function whose body is its argument proves a subtyping by compiling.
- Invariance is the safety net for writes: with a covariant `&mut T` or `Cell<T>`, safe code could store a short-lived borrow where a `'static` one is promised. Reading is covariant, writing contravariant, both invariant.
- Function arguments are contravariant: a callback that accepts more lifetimes is the subtype; a callback that needs `'static` cannot be used for short-lived data.
- The variance of your own types is computed from their fields, and it is part of their interface. An unused parameter is rejected (E0392); `PhantomData` chooses the variance (Chapter 13).
- `&'a mut T<'a>` locks the value: invariance pins `'a` to the value's own lifetime, and the `&mut` borrow must last that long. The shared `&'a T<'a>` is harmless thanks to covariance.

4.13 Exercises

1. Determine the variance in each lifetime parameter of these two structs:

   ```
   struct Transform<'a> { apply: fn(&'a str) -> &'a str }
   struct Lookup<'a, 'b> { keys: &'a [&'b str], hits: Cell<usize> }
   ```

   Confirm each answer with a witness that compiles and one that does not. *Hint:* in `Transform`, `'a` occurs in an argument and in a result. In `Lookup`, ask whether `Cell<usize>` mentions either lifetime.

2. Rewrite `ch04_cell_invariance.rs` with a `RefCell<&'static str>` and `borrow_mut`, and explain, as in the overwrite story, how a covariant `RefCell` would let `main` read a dangling reference. *Hint:* `*view.borrow_mut() = &local;` plays the role of `set`.
3. Given the struct below, can a `Registry<'static>` be passed to a function that expects a `Registry<'short>`?

   ```
   struct Registry<'a> { handlers: Vec<fn(&'a str) -> usize> }
   ```

   Can a `Registry<'short>` be passed where a `Registry<'static>` is expected? Write both witnesses and predict which compiles before you try. *Hint:* `Vec` keeps the variance of its element, and the element is contravariant in `'a`.
4. Write a witness proving that `Option<&'a mut Vec<&'b str>>` is covariant in `'a`, and one showing that it is not covariant in `'b`. Which note does the compiler attach to the second? *Hint:* `Option` is covariant, so it passes on the variances of `&'a mut Vec<&'b str>` unchanged.
5. In `ch04_mut_self_lock.rs`, keep the signature of `add` and remove the second call. Does the program compile? Add a line that only reads `collector.words.len()` after the call, and explain the error with the three steps of *Invariance in practice*. *Hint:* the mutable borrow lasts as long as `'c`, and `'c` covers every later use of `collector`.